PUC Minas

Disciplina: Teste de Software

Refatoração de Testes e Detecção de Test Smells

João Marcelo Carvalho Pereira Araújo

Matrícula: 857649

Repositório: github.com/joaomarcelocpa/test-smelly

1. Análise de Smells

Foram identificados manualmente três smells principais na suíte original (`userService.smelly.test.js`).

Smell	Por que é um mau cheiro	Risco
Lógica condicional (`for` + `if/else` com `expect` dentro) no teste de desativação	O teste passa a ter caminhos de execução próprios; é preciso "executar mentalmente" o teste para saber o que ele verifica, e cada ramo valida um cenário diferente.	Se um ramo não for executado, o `expect` nunca roda e o teste passa em falso. Difícil de depurar e manter.
Teste sem falha garantida (`try/catch` com `expect` apenas no `catch`) no teste de menor de idade	O teste só verifica algo se a exceção for lançada. Se não for, nenhuma asserção é executada.	Se a validação de idade for removida, o teste continua verde e esconde um bug real (falso negativo).
Eager Test / Teste frágil: "criar e buscar" testa dois comportamentos; o teste do relatório compara a linha exata, com `\n` e `startsWith`	Um teste verifica várias responsabilidades, e outro depende do formato exato da saída (detalhe de implementação, que o próprio código diz que pode mudar).	Quando falha, não se sabe qual comportamento quebrou; mudanças cosméticas no relatório quebram o teste sem haver bug.

Também há um **teste ignorado e vazio** (`test.skip`) que dá falsa sensação de cobertura do caso "sem usuários".

2. Processo de Refatoração

Teste escolhido: *"deve desativar usuários se eles não forem administradores"*, o mais problemático (loop, if/else, expects condicionais e dois cenários misturados).

Antes (smelly):

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);
  const todosOsUsuarios = [usuarioComum, usuarioAdmin];
  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois (clean):

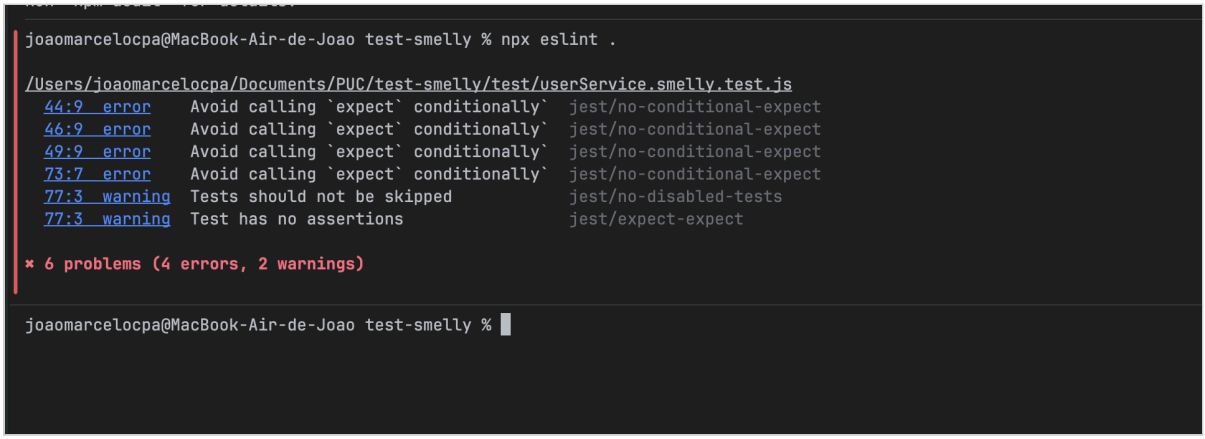
```
describe('deactivateUser', () => {
  test('deve retornar true ao desativar um usuário comum', () => {
    const usuarioComum = criarUsuarioPadrao();
    const resultado = userService.deactivateUser(usuarioComum.id);
    expect(resultado).toBe(true);
  });
  test('deve marcar o usuário comum como inativo após a desativação', () => {
    const usuarioComum = criarUsuarioPadrao();
    userService.deactivateUser(usuarioComum.id);
    const usuarioAtualizado = userService.getUserById(usuarioComum.id);
    expect(usuarioAtualizado.status).toBe('inativo');
  });
  test('deve retornar false ao tentar desativar um administrador', () => {
    const usuarioAdmin = criarUsuarioPadrao(true);
    const resultado = userService.deactivateUser(usuarioAdmin.id);
    expect(resultado).toBe(false);
  });
  test('deve manter o administrador ativo após tentativa de desativação', () => {
    const usuarioAdmin = criarUsuarioPadrao(true);
    userService.deactivateUser(usuarioAdmin.id);
    expect(userService.getUserById(usuarioAdmin.id).status).toBe('ativo');
  });
});
```

Decisões tomadas:

- **Remoção do loop e do if/else:** cada cenário (comum e admin) virou um teste próprio, e todos os `expect` são incondicionais.
- **Padrão AAA:** cada teste tem Arrange (criar usuário), Act (chamar `deactivateUser`) e Assert separados.
- **Um comportamento por teste:** retorno do método e estado final do usuário são verificados separadamente, resolvendo o Eager Test.
- **Nomes descritivos** e um helper `criarUsuarioPadrao` para evitar duplicação.
- Os demais smells também foram corrigidos: `toThrow` no lugar do `try/catch`; relatório verificado por `stringContaining` (id, nome, status) em vez da linha exata; e o teste ignorado foi implementado.

3. Relatório da Ferramenta

Resultado do `npx eslint .` na primeira execução, com a configuração `eslint:recommended` + `plugin:jest/recommended`:



O ESLint detectou automaticamente 4 erros de `jest/no-conditional-expect` (linhas 44, 46, 49 e 73), que correspondem à lógica condicional e ao `try/catch`, e 2 avisos no `test.skip` vazio (`no-disabled-tests` e `expect-expect`). A ferramenta não detecta smells semânticos, como Eager Test, fragilidade do relatório e cobertura faltante; esses só aparecem na análise manual. Após a refatoração,

`userService.clean.test.js` não gera nenhum erro ou aviso, e `npm test` passa (20 testes passando, 1 ignorado no arquivo original).

4. Conclusão

Testes limpos funcionam como documentação executável e só têm valor se puderem falhar quando o código quebra. O trabalho mostrou que uma suíte pode passar e, ainda assim, ser frágil ou enganosa, como no caso do `try/catch`, que esconde a ausência da validação. O padrão AAA, testes focados e nomes claros reduzem o custo de manutenção e facilitam encontrar a causa de uma falha. A análise estática com ESLint complementa a revisão humana: automatiza a detecção dos smells sintáticos, pode rodar no CI e evita que eles voltem. Os smells semânticos continuam dependendo do julgamento do desenvolvedor, então as duas abordagens se completam.